

Deploy the Intern Progress Dashboard on Amazon ECS (Fargate)

Full Implementation Documentation — Capstone

Containerizing my AI-assisted Intern Progress Dashboard and standing up the full ECS/Fargate stack by hand — built end-to-end with every command, output, and error documented, plus the understanding checks I worked through at each stage.

How this doc works. This is a **living document** I fill in *as I build*. Each stage has three things to complete: **(a)** the real command output / proof-of-life, **(b)** my answer to the **Understanding check** questions Kiro poses (this is how I prove I understand what I'm doing, not just that it worked), and **(c)** any errors, logged **verbatim** in the Error Log at the bottom. Placeholders read `<fill in during build>`.

Engineer: Larryking Akere Mokom — AWS Cloud Support Engineer
Intern **Domain:** Amazon ECS · Fargate · DynamoDB · CloudWatch · SNS · CloudFront · Bedrock **Cluster:** `intern-dashboard-cluster` · **Service:** `intern-dashboard-service` · **Region:** `us-east-1` **Account:** `xxxxxxxxxxxx` **App:** Flask + gunicorn, port 8080, health path `/health`, ARM64 image **Platform choice:** ECS on Fargate (chosen over EKS for simplicity + explainability) **Outcome:** `<fill in at the end — what I completed, what met the bar, what raised it>`

Project overview

Goal: take the container image of my locally-built Intern Progress Dashboard and deploy it as a live, load-balanced, self-healing Fargate service that stores state in DynamoDB, logs to CloudWatch, alarms + notifies on failure, scales under load, and serves over HTTPS — then be able to explain the request flow and the common failure modes like a CSE.

Builds on: the app I already built with Kiro (rules-based personalization, a storage layer that swaps in-memory ↔ DynamoDB, and optional Bedrock features that fall back gracefully). Nothing about the app changes here; this is the infra.

Customer lens (Customer Obsession): my "customer" is a CSE intern trying to stay on track to meet the bar. Working back from that, the app must be reliably online (ALB + health checks + self-healing), keep their progress across redeploys (DynamoDB), and stay cheap to run (Fargate/Graviton, on-demand, no NAT).

What is this deployment? (the one-paragraph version)

I am turning one container image into a production-shaped web service on AWS. Instead of running the app on my laptop, ECS runs two copies of it on Fargate (serverless containers — no servers I manage), an Application Load Balancer gives it one stable URL and only sends traffic to healthy copies, DynamoDB holds the data so it survives restarts, CloudWatch watches it, and CloudFront puts it behind HTTPS at the

edge. Every AWS call the app makes uses an IAM **task role**, so there are no passwords or keys anywhere in the code or config.

Mental model

Same restaurant as my ECS Exec exercise, but this time I build the whole thing: the **image** is the sealed franchise recipe kit, **ECR** the warehouse it ships to, the **VPC** the plot of land across two neighborhoods (**AZs**), **public subnets** the storefront, the **IGW** the front door, **security groups** the bouncers, the **ALB** the host stand, the **cluster** the brand, the **task definition** a cook's job spec, a **task** a cook, the **service** the shift manager, the **task role** the cook's ID badge, the **execution role** the loading-dock keycard, **DynamoDB** the pantry that survives every shift, **CloudWatch** the logbook + smoke alarm, **autoscaling** calling in extra cooks at the rush, **SNS** the manager's pager, **CloudFront** citywide pickup windows with a scanner at each, and **Bedrock** the guest chef I phone for specials (who, if he doesn't answer, doesn't stop the standard menu).

How it works under the hood (what I can explain to my mentor)

- **Fargate** runs my container in an AWS-managed micro-VM — I never patch or pay for an idle server. I only declare CPU/memory and the image.
- The **service** maintains my desired count (2). If a task dies or fails its health check, the service launches a replacement — self-healing.

- The **ALB** health-checks `/health` on each task and only routes to healthy ones; a task failing health is drained and replaced without downtime.
- The task authenticates to AWS with its **task role** via the container credentials endpoint (`169.254.170.2`) — short-lived creds, no static keys.
- **DynamoDB** is the durable state; the container filesystem is ephemeral and resets every redeploy, which is exactly why state cannot live there.
- Logs stream through the **awslogs** driver to CloudWatch; an **alarm** on the ALB's `UnHealthyHostCount` publishes to **SNS** → my email.

Does launch type matter? Fargate vs. EC2

I chose **Fargate**: no host to manage, pay only for running tasks, ideal for a light, spiky web app. On the **EC2 launch type** I'd manage an Auto Scaling Group of instances and could bin-pack/Spot for steady workloads — cheaper at constant high utilization but more ops. I note the trade-off deliberately; the EC2 + ASG + capacity-provider variant is my planned stretch.

Credentials & security posture

Identity I build with: `arn:aws:sts::xxxxxxxxxxxx:assumed-role/mokoml/mokoml-Isengard` (verified via `aws sts get-caller-identity`, region `us-east-1`). This is an admin-level sandbox role, acceptable for a personal learning lab. In a real/production context I would use **least-privilege** credentials scoped to only the actions each step needs, not admin — flagging the habit deliberately.

In-app credentials: the app uses **no static keys**. boto3 uses the default credential chain, which on ECS resolves to the **task role**. The task role itself is least-privilege: DynamoDB on one table, Bedrock on one model, SNS on one topic.

Secrets: any sensitive config is injected via **Secrets Manager** (Stage 11), referenced in the task def's secrets block — never pasted into environment or the code. I never paste live credentials into this document.

Stage 0 — Prerequisites (laptop side first)

What / why: prove my laptop is ready (Finch VM, AWS CLI auth) before building anything in AWS, so a local gap never gets misdiagnosed as a cloud problem.

Commands run:

```
finch version
finch vm status
aws sts get-caller-identity --region us-east-1
```

Output / proof:

```
<paste finch version + vm status = Running + caller-identity
account xxxxxxxxxxxx>
```

Understanding check — questions for you: 1. Why do I validate the client/laptop side *before* touching AWS — what did the ECS Exec exercise teach me about this? 2. What is the difference between the **Finch VM** running and a **container** running? (What does `finch ps` being empty tell me?)

My answers: <fill in>

Stage 1 — Build the image + run locally

What / why: build the exact artifact I'll ship and prove it serves /health locally. Golden rule: if it fails locally, it fails on ECS.

Commands run:

```
finch build -t intern-dashboard:local .  
finch run --rm -p 8080:8080 --name intern-dash intern-  
dashboard:local  
curl -s http://localhost:8080/health
```

Output / proof:

```
$ curl -s http://localhost:8080/health  
{"status":"healthy"}  
  
$ curl -s http://localhost:8080/api/coach/me  
{"bedrock_enabled":false, ... , "source":"fallback"}    #  
graceful fallback (Bedrock off locally)
```

```
# UI loaded at http://localhost:8080  
# Build: 15/15 steps FINISHED, tagged intern-dashboard:local;  
context ~9.9MB (.dockerignore trimmed it)
```

Understanding check — questions for you: 1. My Finch VM reports linux/arm64. What image architecture does `finch build` produce by default, and what *must* I set in the Stage 5 task definition to match? What exact error appears if I get this wrong? 2. The Dockerfile is **multi-**

stage, runs as a **non-root** user, and uses **gunicorn** instead of the Flask dev server. Give the one-line reason for each. 3. Locally the app runs with `STORAGE_BACKEND=memory` and Bedrock off. Why is that the right choice for a local test, and what breaks if I relied on memory in production?

My answers (CSE explainers): 1. **Architecture:** *"My image is built for ARM (Apple Silicon), so the Fargate task definition has to be set to ARM64 too — the chip running it has to match the code. If they don't match, the task can't execute the binaries and dies with `exec format error`."* (I set `runtimePlatform.cpuArchitecture = ARM64`; bonus: Graviton is ~20% cheaper.) 2. **Dockerfile:** - Multi-stage → *"I build the app with all the tools in one stage, then copy just the finished result into a clean final image — so the heavy build tools never ship to production, keeping the image small and safe."* - Non-root → *"The container runs as a low-privilege user instead of root, so if someone breaks into the app they land as a 'nobody' with almost no power."* - Gunicorn → *"Flask's built-in server is only for development; gunicorn is the production engine that runs multiple workers so it can handle real traffic without falling over."* 3. **State:** *"Anything saved inside the container gets wiped every time a task restarts or redeploys, so it can't hold real data. That's why progress lives in DynamoDB — an external store that every task shares and that survives restarts."*

CSE read: the local container returning healthy proves the exact artifact I'll ship serves traffic before I spend anything on AWS; the coach endpoint returning source: fallback proves the Bedrock features degrade gracefully instead of erroring when the model is unavailable.

Stage 2 — Push the image to ECR

What / why: store the image in a private registry Fargate can pull from.

Commands run:

```
aws ecr create-repository --repository-name intern-dashboard --region us-east-1
```

```
aws ecr get-login-password --region us-east-1 | finch login --username AWS --password-stdin <ACCT>.dkr.ecr.us-east-1.amazonaws.com
```

```
finch tag intern-dashboard:local <ACCT>.dkr.ecr.us-east-1.amazonaws.com/intern-dashboard:v1
```

```
finch push <ACCT>.dkr.ecr.us-east-1.amazonaws.com/intern-dashboard:v1
```

Output / proof:

```
$ aws ecr create-repository --repository-name intern-dashboard --region us-east-1
```

```
→ repositoryUri: xxxxxxxxxxxx.dkr.ecr.us-east-1.amazonaws.com/intern-dashboard
```

```
$ aws ecr get-login-password ... | finch login ... → Login Succeeded
```

```
$ finch push ...intern-dashboard:v1 → all layers done, 64.7 M pushed (11s)
```

```
(observed: imageTagMutability=MUTABLE, scanOnPush=false – candidate hardening later)
```

Understanding check — questions for you: 1. What does `aws ecr get-login-password | finch login` actually do — why can't I just `finch push` without it? 2. Why tag by version (v1) instead of only latest? How does that help a rollback? 3. ECR is *private*. What would change (security-wise) if it were public, and why does a private repo matter for a real customer?

My answers (CSE explainers): 1. *"get-login-password fetches a short-lived token from AWS, and finch login uses it so ECR trusts me. Without it the registry rejects the push because it has no idea who I am."* 2. *"Tagging v1 gives each build a distinct label, so if v2 breaks in production I roll back by pointing the task definition back at v1. latest is a moving target — you can't tell which build it actually is."* 3. *"A public repo lets anyone pull my image, read the code/config, and hunt for secrets or vulnerabilities. Private keeps it inside my account — only my authenticated identity can pull it."*

Stage 3 — Network foundation (VPC)

What / why: the network the ALB and tasks live in, with rules for what can talk to what.

Built: VPC 10.0.0.0/16; 2 **public** subnets (ALB) + 2 **private** subnets (tasks) across 2 AZs; IGW (public route) + NAT gateway (private route → outbound only); free S3 gateway endpoint; alb-sg (80/443 from world), task-sg (8080 from alb-sg only). Tasks: private subnets, **no public IP**.

Output / proof:

VPC: intern-dashboard-vpc = vpc-0f8402254d78e5d1e
(10.0.0.0/16, us-east-1)

Subnets (4): public1/2 + private1/2 across us-east-1a & us-east-1b

Route tables: intern-dashboard-rtb-public (→ igw),
intern-dashboard-rtb-private1-us-east-1a & -private2-us-east-1b (→ regional NAT)

Gateways/endpoints: intern-dashboard-igw, intern-dashboard-regional-nat,

```
intern-dashboard-vpce-s3 = vpce-0b785a2b4ec8d8666 (S3 gateway),
```

```
intern-dashboard-vpce-dynamodb = vpce-097ac95737141e856 (DynamoDB gateway)
```

```
Security groups: intern-dashboard-alb-sg = sg-0e45a6b3816ca70f0 (inbound 80 from 0.0.0.0/0)
```

```
intern-dashboard-task-sg = sg-06ab3837a1198c1a1 (inbound 8080 from alb-sg only)
```

<paste the 4 subnet IDs when needed at Stage 5>

CSE note (SG-as-source): sourcing task-sg's rule from alb-sg is identity-based, not IP-based — it allows "whatever is the ALB," not an IP range. Since ALB/task IPs are dynamic, the rule keeps working as they change, and only load-balancer traffic can reach tasks on 8080.

Understanding check — questions for you: 1. Why does an ALB require **two** subnets in **two** AZs? What does that buy me? 2. Why put the app tasks in **private** subnets instead of public — what exactly does that protect against? 3. My private tasks have **no public IP**, yet they must pull the image from ECR and reach DynamoDB/Bedrock. How do they get **outbound** access — what does the **NAT gateway** do, and where does it live?

My answers (CSE explainers): 1. "Two AZs = high availability — the ALB stays reachable even if one AZ goes down. It's also a hard requirement: an ALB won't launch in a single AZ." 2. "Private subnets mean the internet can't reach my app directly — the only inbound path is the ALB. The NAT lets my tasks call out, but nobody can call in." 3. "The NAT gateway lives in a public subnet and gives my private tasks outbound-only access. ECS pulls the image with the execution role before the app starts — image layers come via the free S3 gateway endpoint, while ECR API calls + DynamoDB go via the NAT. The running app then uses the task role for DynamoDB/Bedrock." (Correction

learned: image pull = execution role, not task role. PrivateLink/interface endpoints are more *secure*, not cheaper — they cost ~\$7/mo each, so with a NAT already present I'd be paying twice. A DynamoDB *gateway* endpoint is free and optional.)

Stage 4 — State + permissions (DynamoDB + IAM roles)

What / why: durable state + the two roles ECS needs, least-privilege.

Built: table intern-dashboard-progress (PK internId, on-demand); ecsTaskExecutionRole (managed policy); intern-dashboard-task-role (DynamoDB + Bedrock + SNS inline). **Output / proof:**

DynamoDB: intern-dashboard-progress → TableStatus ACTIVE
(PAY_PER_REQUEST, PK internId)

arn:aws:dynamodb:us-east-1:xxxxxxxxxxxx:table/intern-dashboard-progress

Execution role:

arn:aws:iam::xxxxxxxxxxxx:role/ecsTaskExecutionRole (already existed)

Task role: arn:aws:iam::xxxxxxxxxxxx:role/intern-dashboard-task-role

inline policy intern-dashboard-ddb →
GetItem/PutItem/DescribeTable on the table ARN

(Bedrock InvokeModel + SNS Publish added to the task role in later stages.)

Understanding check — questions for you: 1. **The two-badges question (the big one):** what is the difference between the **execution role** and the **task role** — who uses each, and *when*? Which one needs

the DynamoDB permission, and why would putting it on the other one fail? 2. Why must state live in DynamoDB and not the container's filesystem? What exactly happens to filesystem data on a redeploy? 3. I chose **on-demand** capacity over provisioned. What's the frugality argument for my traffic pattern, and when would provisioned be cheaper? 4. My table has only a partition key `internId` (no sort key). Why is that enough for this app's access pattern?

My answers (CSE explainers): 1. *"Execution role sets the container up — ECS uses it to pull the image, push the container's logs to CloudWatch (awslogs driver), and fetch secrets. Task role is what my app does once running — DynamoDB, Bedrock, SNS. DynamoDB went on the task role; on the execution role the app would hit `AccessDeniedException`."* (Correction learned: container-log shipping is the **execution** role's job.) 2. *"State lives in DynamoDB, not the container disk, because the container filesystem is wiped on every redeploy (see Stage 1)."* 3. *"Both DynamoDB modes are equally fast — the choice is traffic shape + cost, not latency. On-demand = pay per request, no idle bill, great for spiky/low intern traffic. Provisioned only wins for steady, high-volume, predictable load."* (Correction learned: it's not about instant-vs-analytics latency.) 4. *"One intern = one item keyed by `internId`, so a partition key finds it directly. I'd add a sort key only if I stored many rows per intern (e.g. a history of snapshots) and needed to sort/range them."*

Stage 5 — Compute (cluster → task definition → service)

What / why: run the container on Fargate with 2 tasks.

Built: cluster intern-dashboard-cluster; task def (0.25 vCPU/0.5GB, **ARM64**, image :v1, port 8080, env incl. STORAGE_BACKEND=dynamodb, both roles, awslogs → /ecs/intern-dashboard); service desired=2 in **private** subnets w/ task-sg, **no public IP. Output / proof:**

Service intern-dashboard-service: status ACTIVE, desired 2, running 2, pending 0

serviceArn: .../cluster/intern-dashboard-cluster/intern-dashboard-service

taskDefinition: intern-dashboard:1 | launchType FARGATE | tasks in private subnets, no public IP

deploymentConfig: rolling, minHealthy 100% / max 200%

(no load balancer yet – attached in Stage 6; not reachable from browser by design)

<optional: paste a gunicorn "Listening at 0.0.0.0:8080" line from `aws logs tail /ecs/intern-dashboard`>

Understanding check — questions for you: 1. Registering a task definition does **not** change my running tasks. Why not, and what step actually rolls the new revision out? 2. What is the difference between a **task definition**, a **task**, and a **service**? (Map each to the restaurant analogy.) 3. If a task shows STOPPED with reason CannotPullContainerError, what are the two most likely causes given my Stage 2/3 choices? 4. BEDROCK_ENABLED=true is set here. If the task role lacked bedrock:InvokeModel, would the app crash, or something else? (Hint: recall the graceful-fallback design.)

My answers (CSE explainers): 1. "Registering a revision doesn't touch running tasks — updating the service to the new revision does. It runs a rolling deployment: launch new-revision tasks, wait for them healthy, then drain the old ones (governed by minHealthy 100% / max 200% = zero downtime). It's proactive, not 'wait for a task to die.'" 2. "Task definition = the blueprint (compute, network mode, env, roles, image,

essential). *Task = one running instance of it. Service = the manager that keeps N healthy tasks running, self-heals failures, rolls out new revisions, and registers tasks with the load balancer.*" 3.

"CannotPullContainerError usually = no network path to ECR (NAT/route misconfig, or a private task with no NAT), or a bad image URI/tag / missing ECR auth. My arch matches and roles are set, so I'd check the private route → NAT first." 4. *"BEDROCK_ENABLED=true without bedrock:InvokeModel wouldn't crash — coach.py catches the AccessDenied and falls back to the rules engine; I'd just see source: fallback + an error in the logs."* (I set it false for now anyway.)

CSE read: two RUNNING tasks prove the image pulls (ARM64 via NAT), the roles work, and private-subnet networking is correct — the whole core is validated *before* I add the ALB. I can't reach it from a browser yet **by design:** no public entry exists until the ALB (Stage 6).

Stage 6 — Load balancer (ALB) → official rubric floor

What / why: a stable public URL that health-checks and load-balances the tasks.

Built: target group (IP, HTTP 8080, health /health); internet-facing ALB in public subnets w/ alb-sg; listener 80 → TG; TG attached to the service. **Output / proof:**

Target group intern-dashboard-tg (.../650b2dd2bee15209): BOTH targets healthy

10.0.133.235 healthy

10.0.144.145 healthy

ALB intern-dashboard-alb (.../416dbe2bdaa2547e), internet-facing, public subnets, alb-sg

Listener :80 → forward → target group

Service now shows loadBalancers = [intern-dashboard-tg]
(attached via update-service)

```
$ curl http://intern-dashboard-alb-1746443957.us-east-1.elb.amazonaws.com/health
```

```
{"status":"healthy"} ← LIVE ON THE INTERNET
```

Understanding check — questions for you: 1. Trace a single request from my browser to a response, naming every hop (DNS → ALB → target group → task → DynamoDB → back). This is the demo-day question. 2. The health check path is /health. What does the ALB *do* with a task that starts returning non-200 on that path, and how does that produce "self-healing"? 3. If the ALB returns **503** with no healthy targets, what are the top three things I'd check, in order?

My answers (CSE explainers): 1. "Browser → (CloudFront →) ALB on :80 → listener forwards to the target group → a healthy Fargate task (registered by IP, since awsvpc/IP mode) on :8080 → app reads/writes DynamoDB via the task role → response back. The ALB health-checks the targets and only routes to healthy ones; the ECS service registers task IPs and replaces failed tasks." 2. "If a task fails the /health check, the ALB stops sending it traffic and drains in-flight connections, while ECS stops the bad task and launches a replacement to keep desired count — that's the self-healing." 3. "An ALB 503 = no healthy targets to route to (not 'server not found'). Check order: (1) target-group health — any healthy targets? (2) if unhealthy, why — /health path/port, task-sg allows ALB on 8080, app logs; (3) is ECS even running tasks — desired vs running, service Events, stopped reasons."

✅ **Rubric checkpoint (official minimum met):** deployed on ECS, reachable via the ALB URL, health check working. ALB URL saved: `http://intern-dashboard-alb-1746443957.us-east-1.elb.amazonaws.com`

Stage 7 — CloudWatch logs + alarm

What / why: confirm logs flow; alarm on unhealthy state → SNS email.

Built: logs at `/ecs/intern-dashboard`; alarm on `UnHealthyHostCount >= 1` → SNS topic `intern-dashboard-alerts` (email subscribed + confirmed). **Output / proof:**

⟨paste alarm in OK; then the ALARM email screenshot from the induced-failure test⟩

Understanding check — questions for you: 1. The logs flow via the `awslogs` driver. Where in the task definition is that configured, and which **role** must permit writing logs — task or execution? 2. My alarm watches `UnHealthyHostCount` on the ALB target group. Why is that a better signal than, say, CPU, for "my app is down"? 3. Walk through the induced-failure test: what did I change, what did the alarm do, and what did I revert?

My answers: ⟨fill in⟩

Stage 8 — Auto-scaling + load test

What / why: scale tasks on load, then prove it with a load test. Scaling is at the **task** level (Service Auto Scaling adjusts the service's `desiredCount`); on Fargate there's no instance layer to manage.

Built (and the journey — a real Dive Deep story): - First tried **target tracking on CPU 50%** — but a load test showed CPU peaked at only ~5.6% (avg ~1%). My app is **I/O-bound, not CPU-bound**, so CPU is the wrong scaling signal and can't be driven high with a light /health endpoint. - Switched to the right metric for a web service:

ALBRequestCountPerTarget. - For a fast, controllable demo I used **step scaling** (two policies + two alarms): - req-high (RequestCountPerTarget ≥ 1000 , 1 datapoint/60s) $\rightarrow$ stepout $\rightarrow$ desired **6** - req-low (< 200 for **10** datapoints/60s, treat-missing=breaching) $\rightarrow$ stepin $\rightarrow$ desired **2** - $\rightarrow$ **fast scale-out (~2 min) + gentle scale-in (~10 min of low traffic).** - min 2 / max 6. Load generator: `hey -z 300s -c 200`.

Output / proof:

Under load: RequestCountPerTarget crossed threshold $\rightarrow$ req-high alarm = ALARM $\rightarrow$

desired 2 $\rightarrow$ 6, running 2 $\rightarrow$ 6 (verified live).

Scale-in eases back to 2.

Note: CPU peaked ~5.6% under 350 req/s $\rightarrow$ proof the app is I/O-bound, not CPU-bound.

Understanding check — questions for you: 1. Service Auto Scaling (tasks) vs ASG/capacity-provider (instances) — which on Fargate? 2. Why request-count, not CPU, for this app? 3. Step scaling vs target tracking — how do their policies/alarms differ?

My answers (CSE explainers): 1. "On Fargate I only scale the task layer (Service Auto Scaling adjusts desiredCount); AWS owns the instances, so there's no ASG/capacity provider. On the EC2 launch type I'd also scale the instance layer via a capacity-provider- driven ASG — both horizontal." 2. "My app is I/O-bound — it peaked at ~5% CPU under heavy load, so CPU is a poor signal. Requests-per-target is what actually reflects load for a web service, so I scale on that. Chasing CPU with

more traffic would be brute-forcing the wrong metric." 3. "Step scaling = two separate policies, each wired to its own CloudWatch alarm (req-high→out, req-low→in) — full control, so I made scale-out fast and scale-in gentle. Target tracking = one policy where AWS auto-creates both alarms — simpler but less control. The ECS service never decides on its own: ALB emits the metric → alarm crosses threshold → Application Auto Scaling policy changes desiredCount → ECS launches/drains tasks." (Also learned: real traffic-driven scaling floors at ~2 min because ALB metrics are 1-minute — no config makes it instant; treat-missing-data decides what 'no traffic' means to the scale-in alarm.)

Stage 9 — SNS reminders (EventBridge-scheduled)

What / why: the app's *accountability* feature — automatically remind interns who have overdue/due-soon items, so nobody has to check manually.

Built: - **Reminder job** (app/reminder.py): scans all interns (storage.scan_all), reuses rules.evaluate to find overdue/due-soon items, publishes a digest to SNS topic intern-dashboard-alerts. Runs as a **run-to-completion Fargate task** using the same image with the command `python -m app.reminder`. - **Dedicated task def** intern-dashboard-reminder (command + SNS_TOPIC_ARN baked in). Task role granted sns:Publish + dynamodb:Scan. - **EventBridge Scheduler** intern-dashboard-daily-reminders (ENABLED, cron(0 17 ? * MON-FRI *), America/Chicago) → RunTask via a scheduler IAM role (ecs:RunTask + iam:PassRole). Manual fire also works for the demo.

Output / proof:

Manual run-task (intern-dashboard-reminder) → exit 0 →
published digest to SNS →

email received listing interns with overdue items (Larry 18,
mokokml 3, me 5).

Schedule: state ENABLED, cron(0 17 ? * MON-FRI *),
America/Chicago.

Understanding check — questions for you: 1. SNS is pub/sub — publisher/subscriber here? How add a 2nd subscriber w/o app change? 2. Which role lets the app publish, and which lets EventBridge run the task? 3. Manual vs scheduled trigger — what makes it automatic?

My answers (CSE explainers): 1. *"Publisher = my reminder task; subscriber = my email on the intern-dashboard-alerts topic. I could add a Lambda or SQS as another subscriber to the topic without touching the app — that's the point of pub/sub decoupling."* 2. *"The task role has sns:Publish (the app publishes) and dynamodb:Scan (to read all interns). A separate scheduler role has ecs:RunTask + iam:PassRole so EventBridge can launch the task and pass it those roles."* 3. *"Nothing's automatic until EventBridge Scheduler runs it — it calls RunTask on a cron (weekdays 5pm CT). The reminder is a run-to-completion task: it starts, scans, publishes, and exits (STOPPED, exit 0) — unlike the web service which runs forever. Two trigger paths now: user requests via the ALB, and scheduled events via EventBridge."*
(Delivery note: SNS sends a digest to topic subscribers; true per-intern email would use SES, which is sandbox-limited here — documented future step.)

Stage 10 — CloudFront (HTTPS/CDN) → meet the bar (my scope) done

What / why: HTTPS + edge caching in front of the ALB.

Built: distribution (Free plan), origin = ALB DNS, **origin protocol HTTP:80** (ALB has no HTTPS listener), **Viewer protocol policy = Redirect HTTP→HTTPS**, allowed methods GET/HEAD/OPTIONS/PUT/POST/PATCH/DELETE, **Cache policy = CachingDisabled**, **Origin request policy = AllViewer**, IPv4-only origin, WAF + rate-limiting (monitor mode). ALB SG then locked to CloudFront's `com.amazonaws.global.cloudfront.origin-facing` prefix list.

Design note — why CachingDisabled (for the doc): this is a **dynamic** app — every response (dashboard state, `/api/*`, `/health`) can differ per intern and per request. If I let CloudFront cache responses, one intern could be served another's cached progress, or a stale dashboard after an update — a correctness bug. `CachingDisabled` forces CloudFront to forward every request to the origin. I still get CloudFront's real benefits here: **HTTPS at the edge, DDoS/WAF protection, and getting the ALB out of the public-endpoint doghouse** (SG locked to CloudFront). Edge *caching* would only matter if I served heavy static assets; if I wanted that later I'd add a separate `/static/*` cache behavior and keep the dynamic paths on `CachingDisabled`. (The console's default "UseOriginCacheControl Headers" policy caches based on `Cache-Control` headers my Flask app doesn't send, which risks unpredictable caching — so I chose `CachingDisabled` explicitly.) **Output / proof:**

Distribution `intern-dashboard-cf` (E5P1CJYHT14ND) → `d1tdk7tryjck3a.cloudfront.net`, Status Deployed

Origin = `intern-dashboard-alb`; ALB SG now allows `:80` from CloudFront prefix list `pl-3b927c52` (+ my IP, temp)

```
$ curl -s https://d1tdk7tryjck3a.cloudfront.net/health
```

`{"status":"healthy"}` ← HTTPS, reachable from anywhere (not just my IP)

Design note — two access paths (kept deliberately): `alb-sg :80` allows (a) **CloudFront's prefix list** = the production path users take (HTTPS + WAF + cache), and (b) **my IP /32** = a restricted **diagnostic / break-glass** path straight to the origin. I keep (b) on purpose for **fault isolation**: if the CloudFront URL fails but the direct ALB works, the problem is at the *edge* (CloudFront/WAF/cache); if both fail, it's the *ALB/ECS/app*. It also lets me test the raw origin without edge caching/WAF interference. Safe because it's a single /32, not `0.0.0.0/0` (so DyePack stays satisfied). This is classic split-half troubleshooting — "is it the CDN or the origin?" — the same technique used on real support cases.

Understanding check — questions for you: 1. Why must I set `/api/*` and `/health` to **no-cache** while caching `/static/*`? What would break if I cached the API responses? 2. How does CloudFront give me working **HTTPS** without me buying a certificate or a domain? 3. What does putting CloudFront in front of the ALB do for (a) latency for a faraway user and (b) load on my ECS tasks?

My answers (CSE explainers): 1. *"My app is dynamic and per-user, so caching would serve stale or someone else's data — `CachingDisabled` sends every request to the origin for fresh, correct responses."* (I still keep CloudFront's HTTPS, WAF, DDoS.) 2. *"CloudFront's default `*.cloudfront.net` domain ships with an AWS-managed TLS cert, so HTTPS works out of the box — no domain or certificate to buy."* 3. *"CloudFront serves users from a nearby edge (lower latency) and shields the origin with edge TLS + WAF/DDoS; with caching on it also offloads ECS — but I disabled caching, so today that offload only kicks in if I cache static assets."*

✅ **Stage 10 (CloudFront/HTTPS) done.** Public HTTPS front door live from anywhere: `https://d1tdk7tryjck3a.cloudfront.net`. Still pending in my scope: observability (7), autoscaling (8), SNS (9).

Stage 11 — Guardrails + Secrets Manager (raise the bar)

What / why: a Bedrock Guardrail (PII masking) over every AI call, and a secret moved out of the task definition.

Built: `guardrail intern-dashboard-guardrail (PII → Mask) + version;`
`task role bedrock:ApplyGuardrail; task-def env`
`BEDROCK_GUARDRAIL_ID/_VERSION;` a secret referenced via the task def secrets block. **Output / proof:**

<paste an email typed into the STAR box returned MASKED + secret injected (not shown in task def env)>

Understanding check — questions for you: 1. My app already redacts nothing itself — the Guardrail does it. Why is applying PII masking at the **Bedrock** layer (both prompt and response) better than trying to strip it in my own code? 2. What is the difference between putting a value in the task def environment vs. secrets? Where does each end up, and who can see it? 3. This is "defense in depth." Name the layers protecting an AI call in my app end to end.

My answers: <fill in>

Authentication & Accounts (Amazon Cognito) — raise the bar

Why: the DyePack finding (Error #4) flagged the app as a *public, unauthenticated* endpoint. Real login fixes that at the root, and gives each intern an account keyed by their **verified email** — which is what makes per-intern persistence (name, start date, progress) and the planned email reminders possible.

What I built: - **Cognito user pool** `us-east-1_L1ypQtQl0`: self-registration, email sign-in, required **name** attribute; confidential app client `obecuct1e2du34nv45md4ltaf` (auth-code grant, scopes `openid/email/profile`); managed-login domain `https://us-east-1l1ypqtql0.auth.us-east-1.amazoncognito.com`. - **App-level OIDC** in Flask via Authlib (`/login` → `/callback` → `/logout`), feature-flagged by `AUTH_ENABLED` (off locally = dev identity; on in ECS = real login). A `before_request` gate bounces unauthenticated users to login (except `/health`, `/static`, auth routes) — this removes the unauthenticated endpoint. Identity = the verified email; the URL param is ignored so nobody can load another intern's data. - **Peccy landing page** (`login.html`) for logged-out visitors; the dashboard (with the intern's **name** in the header + Log out) after login. - **Secrets in Secrets Manager** (tagged `auto-delete=no` vs Spring Clean): `intern-dashboard/flask-secret` (session signing key) + `intern-dashboard/cognito-client-secret` (app client secret). The **execution role** has `secretsmanager:GetSecretValue` on `intern-dashboard/*`; the task def references them under `secrets`: (never plaintext env). - Shipped as image **v2** / task def **:2** (also flipped `BEDROCK_ENABLED=true` + granted the task role `bedrock:InvokeModel` so the Coach is live).

Output / proof:

Live: `https://d1tdk7tryjck3a.cloudfront.net` → Peccy landing → Cognito login →

 dashboard shows my name. `/api/me` returns my Cognito email + name.

```
/api/health/services → "bedrock": {"up": true}.
```

Passwords / secrets mental model (say to mentor): user passwords live in **Cognito** (hashed/managed — I never see them); my app's own secrets (session key, client secret) live in **Secrets Manager** (encrypted, injected at runtime by the execution role); **DynamoDB** holds intern data only. Three stores, three jobs.

Bedrock is an API, not a container (say to mentor): the Coach isn't hosted in my app — my Fargate container makes an authenticated `InvokeModel/Converse` call to the managed Bedrock service via the **task role** (same pattern as DynamoDB). The model runs on AWS's fleet, so it adds **zero compute** to my task. It shows "up" only when all three are true — `BEDROCK_ENABLED=true`, task-role `bedrock:InvokeModel`, and Nova **model access** enabled — and the health map does a real 1-token ping (cached 60s) to prove reachability.

Understanding checks (answers): 1. *App-level OIDC over rolling my own passwords?* — Cognito owns passwords, verification, and reset; I only write the redirect/callback glue, so the riskiest security code isn't mine. 2. *Identity from the session email, not the URL?* — so an intern can't load another's data by editing the URL; the server trusts only the verified token. 3. *What did auth fix from the security finding?* — the endpoint is no longer publicly unauthenticated; every page requires a Cognito login.

Stretch — DONE

B) Weekly cohort analytics on Fargate Spot  - `app/analytics.py`: scans all interns (`storage.scan_all`), reuses `rules.evaluate` to compute cohort stats (avg completion, on-track/at-risk/behind, most-overdue items), logs a report + publishes a summary to SNS. A **run-to-**

completion batch task (intern-dashboard-analytics), run with `--capacity-provider-strategy capacityProvider=FARGATE_SPOT`. -

Why Fargate Spot over EC2 Spot: same ~70% Spot discount + interruption tolerance, but **zero instance management** (no AMI/ASG/launch template) and **no idle instance** between weekly runs — cheaper in *total* cost (dollars + ops). EC2 Spot only wins for specific instance types or very high sustained utilization; this job is small/periodic/generic. Two-capacity-provider story: **FARGATE** for user-facing (web + reminders), **FARGATE_SPOT** for background analytics. - Verified: ran on FARGATE_SPOT, `exit 0`, report — "Interns 3, avg 79%, on-track 2 / behind 1."

A) ADOT observability sidecar ✅ (mentor's logging+metrics ask) - `app/telemetry.py`: **flag-gated (OTEL_ENABLED) + fully guarded** OTel setup — emits a custom `http_requests` counter (path+status) via OTLP/HTTP to the sidecar. No-op if disabled or if export fails, so it can never break the web app. - Task def :4 adds a 2nd container `aws-otel-collector` (**essential:false**, depends on the app at START), config via `AOT_CONFIG_CONTENT` (OTLP receiver → `awsemf` exporter → CloudWatch namespace `InternDashboard`). Task CPU/mem bumped 256/512 → 512/1024 for two containers. Task role granted CloudWatch logs + `PutMetricData`. - Verified: both containers RUNNING; `http_requests` metric present in the `InternDashboard` namespace in CloudWatch. - Sidecar concept: cross-cutting telemetry runs *beside* the app in the same task; `essential:false` means its failure never takes down the app.

Also available to explain (not built): EC2+ASG capacity provider (the manual launch-type path), EKS second platform, AWS SAA cert.

Error log (verbatim, with root cause and fix)

Copy each error **exactly** as it appeared. Paraphrasing loses the detail a CSE needs. One row per distinct error.

1

FATA[0000] context needs to be specified

1

Ran `finch build -t intern-dashboard:local` with no build-context argument, and from `~` instead of the repo

`cd` into the repo and append `.` (`finch build -t intern-dashboard:local .`) — the `.` tells Finch which folder to build from / where the Dockerfile is

2

InvalidRouteTableId.Malformed ... Invalid Id: 'rtb-0698...
rtb-0a33...'

3

`zsh` (macOS default) doesn't word-split unquoted variables like `bash`, so `$PRIV_RTS` (two IDs) was passed as one argument

Pass the two route-table IDs explicitly as separate args (or use `${=PRIV_RTS}` in `zsh`)

3

⟨paste⟩

⟨#⟩

⟨fill in⟩

<fill in>

3

shell hung at quote> (heredoc never terminated)

4

Pasted heredoc had an **indented** EOF; the terminator is only recognized at column 0, so zsh stayed inside the heredoc waiting for input

Ctrl-C to escape, then pass the IAM JSON **inline** as single-quoted one-line arguments instead of heredocs

4

App unreachable; Epoxy elb_listener_delete auto-mitigation on eni/port-80 (DyePack finding palisade_dyepack_ec2_ip_authentication)

6

Personal Isengard guardrail: ALB was open to 0.0.0.0/0 on :80 (public, unauthenticated ENI endpoint); AWS security auto-deleted the listener

Root cause = ALB SG world-open. Immediate: restrict alb-sg :80 to my IP /32 + recreate listener. End state (Stage 10): CloudFront (HTTPS) in front, and lock alb-sg to the com.amazonaws.global.cloudfront.origin-facing prefix list so only CloudFront reaches the ALB. This ec2/ENI-IP detector is satisfied by the SG lockdown; CloudFront doesn't add auth, so confirmed approach with mentor.

...

How this maps to the rubric

1. Deployment

Exceeds

Deployed on ECS/Fargate, ALB+HTTPS URL, health check (meets); + **autoscaling, SNS, alarm, CloudFront, Guardrails/Secrets** (exceeds). URL: <paste>

2. Understanding

Exceeds

Request-flow narration (Stage 6 Q1); trade-offs: **ECS vs EKS, Fargate vs EC2, public subnet vs NAT, on-demand vs provisioned DynamoDB**; "what fails first" table. Answers filled in per stage.

3. Demo

Exceeds

Live request over the URL; **kill a task** → **show ECS self-heal**; **hey load test** → **show autoscale**; **masked email** via Guardrail. <note what I demoed>

Leadership Principles evidence: - **Customer Obsession:** built for the intern-staying-on-track customer; reliable, persistent, cheap. - **Dive Deep:** can explain every hop and every failure mode at the AWS-API level. - **Frugality:** Fargate/Graviton, on-demand DynamoDB, no NAT, no custom domain, token-capped Bedrock — each a deliberate, explained trade-off. - **Ownership:** built all infrastructure by hand end-to-end. - **Learn & Be Curious:** chose to add Bedrock + Guardrails beyond the minimum.

Demo-day cheat sheet

- **Request flow:** browser → CloudFront (TLS + cache /static) → ALB (/health target group) → Fargate task :8080 → DynamoDB via task role (no static creds) → response; logs → CloudWatch; alarm → SNS.

- **"What breaks first?"** ALB target health → ECS service **Events** → CloudWatch logs. AccessDenied = task role; exec format error = arch mismatch; 503 = no healthy targets.

- **Exceeds moves:** kill a task (self-heal), hey load test (autoscale), masked email (Guardrail).